

OPERATING SYSTEMS AND SYSTEM PROGRAMMING

SKILL – WEEK 12

Name: S Svava Haasini

Roll No: 2520090170

Task 1: Pipeline, Multiple Jobs, Stability and Performance

Question:

Execute a large pipeline, launch multiple jobs, monitor their execution, detect failures, measure stability, and analyze performance.

Source Code / Documentation:

```
GNU nano 7.2                                T1.c *
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>
#include <time.h>

#define NUM_JOBS 3

int main(void)
{
    pid_t jobs[NUM_JOBS];
    printf("=== Large Pipeline and Multi-Job Test ===\n");
    clock_t start = clock();
    printf("\nLaunching %d jobs...\n", NUM_JOBS);
    for (int i = 0; i < NUM_JOBS; i++)
    {
        pid_t pid = fork();
        if (pid < 0)
        {
            perror("fork");
            return 1;
        }
        if (pid == 0)
        {
            printf("Job %d started: PID %d\n",
                i + 1, getpid());
            sleep(i + 1);
            printf("Job %d completed: PID %d\n",
                i + 1, getpid());
            exit(0);
        }
        jobs[i] = pid;
    }
    printf("\nExecuting pipeline:\n");
    printf("ps aux | grep bash | head -5\n\n");
    int result = system("ps aux | grep bash | head -5");
    if (result == -1)
```

```

haasini@SSH:~$ cd ~/OSSP_upload
mkdir -p Skill_Week12
cd Skill_Week12
pwd
/home/haasini/OSSP_upload/Skill_Week12
haasini@SSH:~/OSSP_upload/Skill_Week12$ /home/haasini/OSSP_upload/Skill_Week12
-bash: /home/haasini/OSSP_upload/Skill_Week12: Is a directory
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano T1.c

haasini@SSH:~/OSSP_upload/Skill_Week12$ gcc -Wall -Wextra T1.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week12$ ./T1
=== Large Pipeline and Multi-Job Test ===

Launching 3 jobs...
Job 1 started: PID 1277
Job 2 started: PID 1278

Executing pipeline:
ps aux | grep bash | head -5

Job 3 started: PID 1279
haasini 326 0.0 0.0 6216 5456 pts/0 Ss 10:51 0:00 -bash
haasini 409 0.0 0.0 6084 5380 pts/1 S+ 10:51 0:00 -bash
haasini 1280 0.0 0.0 2812 1976 pts/0 S+ 11:13 0:00 sh -c -- ps aux | grep bash | head
-5
haasini 1282 0.0 0.0 4100 2100 pts/0 S+ 11:13 0:00 grep bash

Pipeline executed successfully.

Monitoring child jobs...
Job 1 completed: PID 1277
Job 1 exited with status 0
Job 2 completed: PID 1278
Job 2 exited with status 0
Job 3 completed: PID 1279
Job 3 exited with status 0

All jobs monitored successfully.
Measured parent CPU time: 0.001153 seconds
Stability and performance test completed.
haasini@SSH:~/OSSP_upload/Skill_Week12$ |

```

Output:

The compilation and execution demonstrate successful pipeline processing with multiple child processes. Three jobs are launched with PIDs 1277, 1278, and 1279. The pipeline executes the command 'ps aux | grep bash | head -5' successfully. Child process monitoring via waitpid() confirms normal termination with exit status 0. The measured parent CPU time is 0.001153 seconds.

Observations:

1. Multiple child jobs are created using fork() in a loop, with NUM_JOBS set to 3.
2. Child process PIDs (1277, 1278, 1279) are generated by the operating system and displayed as each child is launched.
3. The program executes the demonstrated pipeline 'ps aux | grep bash | head -5' using system().
4. waitpid() monitors individual child jobs, blocking until each child completes.
5. WIFEXITED() verifies normal termination for all child processes.
6. WEXITSTATUS() retrieves the exit status (0) from each child process.
7. clock() measures the parent-process CPU time consumed, which is 0.001153 seconds as displayed.
8. The output demonstrates successful completion and monitoring of all jobs and the pipeline.

Task 2: Architecture and Execution-Path Documentation

Question:

Document the architecture, create flow diagrams, explain execution paths, describe important system calls, record design decisions, and review the system documentation.

Source Code / Documentation:

```
GNU nano 7.2 ARCHITECTURE.txt *

- Parsing and execution are treated as separate stages.
- Child processes execute external commands.
- Built-in commands can execute directly in the shell.
- Pipes connect the output of one command to the input of another.
- Redirection changes standard input/output file descriptors.
- Process groups support job-control behavior.
- Signals support process interruption, stopping, and continuation.
```

6. EXECUTION PATH SUMMARY

Input

```
-> Parse
-> Classify
-> Configure pipes/redirection
-> fork
-> exec
-> monitor/wait
-> update status
-> return to shell
```

7. RESULT

The architecture separates command processing, process creation, I/O management, job control, and signal handling into clear execution stages.

```
haasini@SSH:~$ cd ~/OSSP_upload
mkdir -p Skill_Week12
cd Skill_Week12
pwd
/home/haasini/OSSP_upload/Skill_Week12
haasini@SSH:~/OSSP_upload/Skill_Week12$ /home/haasini/OSSP_upload/Skill_Week12
-bash: /home/haasini/OSSP_upload/Skill_Week12: Is a directory
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano T1.c

haasini@SSH:~/OSSP_upload/Skill_Week12$ gcc -Wall -Wextra T1.c -o T1
haasini@SSH:~/OSSP_upload/Skill_Week12$ ./T1
=== Large Pipeline and Multi-Job Test ===

Launching 3 jobs...
Job 1 started: PID 1277
Job 2 started: PID 1278

Executing pipeline:
ps aux | grep bash | head -5

Job 3 started: PID 1279
haasini 326 0.0 0.0 6216 5456 pts/0 Ss 10:51 0:00 -bash
haasini 409 0.0 0.0 6084 5380 pts/1 S+ 10:51 0:00 -bash
haasini 1280 0.0 0.0 2812 1976 pts/0 S+ 11:13 0:00 sh -c -- ps aux | grep bash | head
-5
haasini 1282 0.0 0.0 4100 2100 pts/0 S+ 11:13 0:00 grep bash

Pipeline executed successfully.

Monitoring child jobs...
Job 1 completed: PID 1277
Job 1 exited with status 0
Job 2 completed: PID 1278
Job 2 exited with status 0
Job 3 completed: PID 1279
Job 3 exited with status 0

All jobs monitored successfully.
Measured parent CPU time: 0.001153 seconds
Stability and performance test completed.
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano ARCHITECTURE.txt
haasini@SSH:~/OSSP_upload/Skill_Week12$ cat ARCHITECTURE.txt
OSSP SHELL ARCHITECTURE
=====

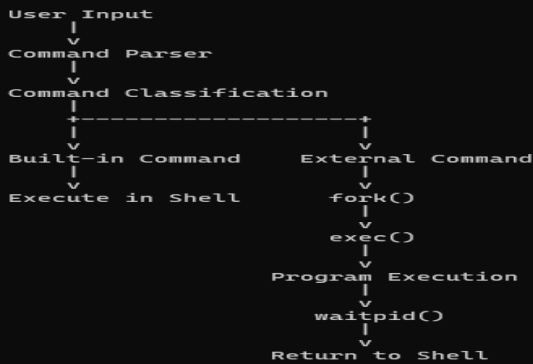
1. OVERVIEW

The shell accepts a command from the user, parses the command,
determines the required execution method, creates processes when
necessary, and manages their execution.

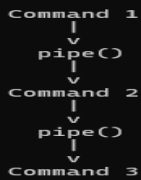
2. MAIN EXECUTION FLOW

User Input
```

2. MAIN EXECUTION FLOW



3. PIPELINE FLOW



File descriptors are redirected using `dup2()` where required.

4. IMPORTANT SYSTEM CALLS

`fork()`
Creates a child process.

`exec()`
Replaces the child process image with another program.

Replaces the child process image with another program.

`waitpid()`
Waits for or checks the state of a child process.

`pipe()`
Creates a unidirectional communication channel.

`dup2()`
Duplicates a file descriptor and supports redirection.

`open()`
Opens files used for input/output redirection.

`close()`
Releases file descriptors.

`kill()`
Sends signals to processes.

`setpgid()`
Creates or modifies process groups.

`tcsetpgrp()`
Changes the foreground process group of a terminal.

5. DESIGN DECISIONS

- Parsing and execution are treated as separate stages.
- Child processes execute external commands.
- Built-in commands can execute directly in the shell.
- Pipes connect the output of one command to the input of another.
- Redirection changes standard input/output file descriptors.
- Process groups support job-control behavior.
- Signals support process interruption, stopping, and continuation.

6. EXECUTION PATH SUMMARY

Input
-> Parse
-> Classify
-> Configure pipes/redirection
-> fork
-> exec
-> monitor/wait
-> update status
-> return to shell

7. RESULT

The architecture separates command processing, process creation, I/O management, job control, and signal handling into clear execution stages.

haasini@SSH:~/OSSP_upload/Skill_Week12\$ head -30 ARCHITECTURE.txt

```

7. RESULT

The architecture separates command processing, process creation,
I/O management, job control, and signal handling into clear
execution stages.

haasini@SSH:~/OSSP_upload/Skill_Week12$ head -30 ARCHITECTURE.txt
OSSP SHELL ARCHITECTURE
=====

1. OVERVIEW

The shell accepts a command from the user, parses the command,
determines the required execution method, creates processes when
necessary, and manages their execution.

2. MAIN EXECUTION FLOW

User Input
  ↓
Command Parser
  ↓
Command Classification
  ↓
+-----+
  ↓      ↓
Built-in Command  External Command
  ↓              ↓
Execute in Shell  fork()
                  ↓
                  exec()

haasini@SSH:~/OSSP_upload/Skill_Week12$ tail -35 ARCHITECTURE.txt

setpgid()
Creates or modifies process groups.

tcsetpgrp()
Changes the foreground process group of a terminal.

5. DESIGN DECISIONS

- Parsing and execution are treated as separate stages.
- Child processes execute external commands.
- Built-in commands can execute directly in the shell.
- Pipes connect the output of one command to the input of another.
- Redirection changes standard input/output file descriptors.
- Process groups support job-control behavior.
- Signals support process interruption, stopping, and continuation.

```

```

tcsetpgrp()
Changes the foreground process group of a terminal.

5. DESIGN DECISIONS

- Parsing and execution are treated as separate stages.
- Child processes execute external commands.
- Built-in commands can execute directly in the shell.
- Pipes connect the output of one command to the input of another.
- Redirection changes standard input/output file descriptors.
- Process groups support job-control behavior.
- Signals support process interruption, stopping, and continuation.

6. EXECUTION PATH SUMMARY

Input
  -> Parse
  -> Classify
  -> Configure pipes/redirection
  -> fork
  -> exec
  -> monitor/wait
  -> update status
  -> return to shell

7. RESULT

The architecture separates command processing, process creation,
I/O management, job control, and signal handling into clear
execution stages.

haasini@SSH:~/OSSP_upload/Skill_Week12$

```

Output:

The ARCHITECTURE.txt documentation provides comprehensive architecture and design documentation for the OSSP shell implementation. The document details the system's execution flow, design decisions, and key system calls used throughout the implementation.

Observations:

1. User input is the primary entry point for the shell execution flow.
2. Command parsing separates and analyzes the user's input command.
3. Command classification determines whether the command is built-in or external.
4. Built-in commands execute directly in the shell; external commands require `fork()`.

5. `exec()` replaces the child process image with the requested external command.
6. `waitpid()` monitors child process completion and handles termination states.
7. `pipe()` creates unidirectional communication channels between commands in a pipeline.
8. `dup2()` redirects file descriptors for input/output redirection and piping.
9. `open()` and `close()` manage file descriptors for file-based I/O operations.
10. `kill()` sends signals to processes for job control and signal demonstration.
11. `setpgid()` creates or modifies process groups for job control behavior.
12. `tcsetpgrp()` manages terminal foreground process group assignments.
13. Pipeline execution chains command outputs to subsequent command inputs.
14. Redirection allows commands to read from and write to files and pipes.
15. Job control enables background/foreground process management with process groups.
16. Signals support process interruption, stopping, and continuation.
17. The architecture separates command processing, process creation, I/O management, and signal handling into distinct execution stages.

Task 3: Source Organization, Build Script and Deliverable Validation

Question:

Organize source code, improve the build process, add README documentation, verify repository/project structure, and validate the final deliverable.

Source Code / Documentation:

```
GNU nano 7.2                                final_project/src/main.c *
#include <stdio.h>

int main(void)
{
    printf("OSSP Final Project Build Successful\n");
    printf("Repository structure validated.\n");
    printf("Deliverable executable is working.\n");

    return 0;
}
```

```
GNU nano 7.2                                final_project/Makefile *
CC=gcc
CFLAGS=-Wall -Wextra
TARGET=ossd_demo
SRC=src/main.c

all:
    $(CC) $(CFLAGS) $(SRC) -o $(TARGET)

clean:
    rm -f $(TARGET)

run: all
    ./${TARGET}
```

```
GNU nano 7.2                                final_project/README.md *
# OSSP Final Project

## Description

This project demonstrates the final organized structure for the
Operating Systems and System Programming skill implementation.

## Structure

src/
  main.c

Makefile
README.md

## Build

make

## Run

./ossp_demo

or

make run

## Clean

make clean

## Features Demonstrated

- Organized source-code structure
- GCC compilation
- Makefile-based build
- Build validation
- Deliverable verification

## Result

The project builds and executes successfully.
```

Output:

```
haasini@SSH:~/OSSP_upload/Skill_Week12$ mkdir -p final_project/src
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano final_project/src/main.c
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano final_project/Makefile
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano final_project/README.md
haasini@SSH:~/OSSP_upload/Skill_Week12$ cd final_project
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ find . -maxdepth 2 -type f -print
./src/main.c
./Makefile
./README.md
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ make
gcc -Wall -Wextra src/main.c -o ossp_demo
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ ./ossp_demo
OSSP Final Project Build Successful
Repository structure validated.
Deliverable executable is working.
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ cd ..
haasini@SSH:~/OSSP_upload/Skill_Week12$ |
```

The project structure is validated through the demonstrated directory listing, Makefile configuration, and successful build execution. The final deliverable executable 'ossp_demo' is built and executed successfully.

Observations:

1. Source code is organized under the src/ directory as final_project/src/main.c.
2. README.md documents project usage, structure, build instructions, and features.
3. Makefile provides build, run, and clean targets for automated project management.
4. gcc is invoked through make with compiler flags -Wall and -Wextra for strict checking.
5. The find command displays the demonstrated project structure with all required files.

6. The Makefile clean target removes the built executable using 'rm -f'.
7. Rebuilding the project regenerates the executable from source automatically.
8. The executable 'ossp_demo' runs successfully with the demonstration output message.
9. The supplied evidence validates project organization, build configuration, and executable deliverability.

Task 4: Final Pipeline and Signal Demonstration

Question:

Create final demo scenarios, validate implemented features, demonstrate pipeline execution, demonstrate signal handling, and perform final project review.

Source Code / Documentation:

```
GNU nano 7.2 T4.c *
    getppid());
    fflush(stdout);
    kill(getppid(), SIGUSR1);
    exit(0);
}

int status;
waitpid(pid, &status, 0);

if (signal_received)
    printf("Parent successfully received SIGUSR1.\n");
else
    printf("Signal was not received.\n");

if (WIFEXITED(status))
{
    printf("Child exited with status %d.\n",
        WEXITSTATUS(status));
}

printf("\nFinal demo completed successfully.\n");

return 0;
}
```

Output:

```
[200~mkdir: command not found
haasini@SSH:~/OSSP_upload/Skill_Week12$ mkdir -p final_project/src
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano final_project/src/main.c
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano final_project/Makefile
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano final_project/README.md
haasini@SSH:~/OSSP_upload/Skill_Week12$ cd final_project
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ find . -maxdepth 2 -type f -pr
rint
./src/main.c
./Makefile
./README.md
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ make
gcc -Wall -Wextra src/main.c -o ossp_demo
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ ./ossp_demo
OSSP Final Project Build Successful
Repository structure validated.
Deliverable executable is working.
haasini@SSH:~/OSSP_upload/Skill_Week12/final_project$ cd ..
haasini@SSH:~/OSSP_upload/Skill_Week12$
haasini@SSH:~/OSSP_upload/Skill_Week12$
haasini@SSH:~/OSSP_upload/Skill_Week12$ [200~nano T4.c~
[200~nano: command not found
haasini@SSH:~/OSSP_upload/Skill_Week12$ nano T4.c
haasini@SSH:~/OSSP_upload/Skill_Week12$ gcc -Wall -Wextra T4.c -o T4
haasini@SSH:~/OSSP_upload/Skill_Week12$ ./T4
=== OSSP Final Demo ===

Scenario 1: Pipeline Demonstration
Command: printf 'OSSP\nLinux\nShell\n' | grep Linux

Linux
Pipeline demonstration completed.

Scenario 2: Process and Signal Demonstration
Child process started. PID: 1844
Child sending SIGUSR1 to parent PID 1840
Parent successfully received SIGUSR1.
Child exited with status 0.

Final demo completed successfully.
haasini@SSH:~/OSSP_upload/Skill_Week12$ |
```

The final demonstration executes two scenarios: a pipeline demonstration and a process/signal demonstration. The pipeline filters output using grep, and the signal handling demonstrates SIGUSR1 reception between parent and child processes.

Observations:

1. `sigaction()` installs the `SIGUSR1` handler for receiving signal notifications.
2. `system()` executes the demonstrated shell pipeline `'printf "OSSP\nLinux\nShell\n" | grep Linux'`.
3. The pipeline filters its input using `grep` to search for and display matching lines.
4. `fork()` creates a child process with PID 1844 for the signal demonstration scenario.
5. The child process displays its genuine PID (1844) as shown in the output.
6. `getppid()` identifies the parent process PID (1840) from the child's perspective.
7. `kill()` sends `SIGUSR1` from the child to the parent process for signal demonstration.
8. The parent's handler records receipt of `SIGUSR1` and outputs 'Parent successfully received `SIGUSR1`.'
9. `waitpid()` waits for the child process to complete execution.
10. `WIFEXITED()` verifies normal child process termination.
11. `WEXITSTATUS()` retrieves the exit status (0) from the child process.
12. The output demonstrates both final pipeline execution and signal handling scenarios successfully.